지난 발표 "Codex/Claude Code 문서로 보는 Harness Engineering" 후속편

# 콘텐츠 생성 자동화 사례로 보는 하네스 엔지니어링 실전

## 코드 기반 에이전트 vs 하네스 기반 에이전트, 뭐가 다른가

발표자: 김혜정 (molly.ouo)

스터디: GAI 스터디

날짜: 2026.04.16

# 지난 발표: "하네스란 무엇인가"

## 지난 발표 요약

- **AI 흐름의 변화**

  프롬프트 엔지니어링 → 오케스트레이션과 구조 선택의 중요성 부각 → 에이전트 등장 · 상태 관리 · 도구 조합 · 평가 루프 설계 → 직접 수행보다 환경 · 규칙 · 피드백 구조 설계

- **하네스**란 특정 일을 잘 수행하기 위해 어떻게 실행·통제할지(운영)에 대해 설계·관리하는 것
  - 지침서(`CLAUDE.md`, `AGENTS.md`), MCP, Skills, Agent 등을 조합

**오늘의 질문**

**같은 "콘텐츠 생성 자동화"를**
**코드 기반으로 구성하는 것과 하네스 기반으로 구성하는 것의**
**차이는 무엇일까?**

# 같은 "콘텐츠 자동화", 완전히 다른 접근

| 구분 | 사례 A (jay.pm.ai) | 사례 B (zoey.day) |
|---|---|---|
| 도메인 | 카드뉴스 자동화 | 상세페이지 현지화 |
| 입력 / 출력 | 페르소나 + 욕구 → 카드뉴스 5장 세트 | Amazon 제품 페이지 → 다국어 상세페이지 |
| 접근 방식 | 코드 기반 (Python) | 하네스 기반 (.md) |
| 핵심 구조 | `*_agent.py` × 4 + `orchestrator.py` | 에이전트 6개(.md) + 스킬 6개 + MCP |
| 외부 연동 | gspread / OpenAI 직접 호출 | MCP 서버 경유 |
| 품질 보장 | 자기 검증 루프 | 독립 QA 에이전트 |

# 7가지 관점으로 본 두 방식의 차이

| 관점 | jay.pm.ai (코드 기반) | zoey.day (하네스 기반) |
|---|---|---|
| 에이전트 정의 | Python 파일이 곧 에이전트 | .md로 역할만 정의, 코드는 런타임에 작성 |
| 프롬프트 관리 | prompts/ 폴더에 텍스트 분리 | skills/references/에 통합 (Progressive) |
| 🔥 품질 보장 | 자기 검증 루프 | QA 에이전트가 독립 검수 |
| 확장성 | 새 기능 = 새 .py + orchestrator 수정 | 새 기능 = 새 .md + 오케스트레이터 스킬 수정 |
| 🔥 외부 연동 | gspread/openai 직접 호출 | MCP 서버로 추상화 |
| 비개발자 수정 | 코드를 읽어야 가능 | 마크다운만 편집하면 됨 |
| 🔥 코드 관리 | scripts/ 유지 (의존성·버전·버그) | 코드 파일 0개 |

🔥 = 주요 차이

# "에이전트를 만든다"는 표현이 의미하는 차이

thumbnail_agent.py (발췌) Python

```python
def generate_thumbnail(persona, desire):
    prompt = load_prompt("thumbnail.txt").format(
        persona=persona, desire=desire
    )
    resp = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}]
    )
    copy = resp.choices[0].message.content
    if not validate_rules(copy):
        return retry_with_feedback(copy)
    return copy
```

.claude/agents/thumbnail-writer.md Markdown

```markdown
## 역할
5개 마케팅 앵글별 후킹 카피를 작성한다.

## 사용 스킬
- generate-thumbnail: 앵글별 헤드라인 3줄 + 칩태그 2개
- evaluate-copy: 규칙 검증 + 자기 평가 (10점)

## 품질 기준
7점 미만이면 최대 3회 재작성
```

CLAUDE.md + agents/*.md + skills/ + config.yaml

같은 일을 하는 두 레포. 우측(하네스)은 실행 가능한 Python 파일이 0개.

**코드 기반** = 내가 직접 코드를 짜서 **에이전트를 만드는 것**

**하네스 기반** = 에이전트가 일할 **환경과 규칙을 설계하는 것**

---

지난 발표에서 말한 "인간의 역할이 '직접 수행'에서 '환경·규칙·피드백 구조 설계'로 이동"이
실제로 어떻게 나타나는지를 보여주는 구체적인 사례.

# 재구성 결과를 세 가지 관점으로 소개

## 프로젝트 구조

```
card-news-agent/   ← 코드 0개
├── CLAUDE.md      ← 진입점
├── config.yaml
├── .claude/
│   ├── agents/    (3개 .md)
│   │   ├── thumbnail-writer
│   │   ├── body-writer
│   │   └── image-renderer
│   └── skills/    (5개)
│       ├── orchestrator
│       ├── generate-thumbnail
│       ├── evaluate-copy 🔥
│       └── generate-image
```

## 기존과의 차이점

- 기존 Python 기반 에이전트는 **검증 로직이 에이전트마다 코드로 중복 선언**
- 하네스 방식은 `evaluate-copy`라는 `SKILL`을 정의해 **썸네일 작성자와 본문 작성자가 동일한 스킬을 공유하여 검증**

## 전체 흐름

```
페르소나 + 욕구 입력
(클로드코드에서 "카드 뉴스 만들어줘 --persona '20대 대학생' --desire 'AI를 더 잘 활용하고 싶다.'")
        |
        ▼
  thumbnail-writer     5개 앵글별 헤드라인 생성 + 자기 검증 루프
        |
  [구글시트 승인 대기]  ← Human-in-the-loop (--auto 로 건너뛰기)
        |
        ▼
   body-writer         본문 3장 + CTA 생성 + 톤 일관성 검증
        |
  [구글시트 승인 대기]
        |
        ▼
  image-renderer       Gemini 배경이미지 + 카드 5장 PNG 렌더링
        |
        ▼
  output/{날짜}_{페르소나}/
  ├── 01_thumbnail.png    ├── 03_body2.png    └── 05_cta.png
  ├── 02_body1.png        └── 04_body3.png
```

# Agent Engineering 5가지 설계 포인트

| 설계 포인트 | 이번 사례 구현 |
|---|---|
| **1) 작업 단위** | 페르소나+욕구 1쌍 = 1 세트 (썸네일 · 본문 3 · CTA)<br>산출물과 로그는 `output/{날짜}_{페르소나}/` 폴더에 함께 묶어 관리 |
| **2) 실행 권한** | 에이전트가 로컬에서 `pip install` · Playwright · 파일 I/O 까지 자동 수행<br>외부 시스템(구글 시트)은 **서비스 계정**으로 제한된 권한만 허용 |
| **3) 분업** | 각 작업을 가장 잘하는 도구에 위임<br>· **카피 생성 · 규칙 검증 · 품질 평가** → Claude (언어 판단)<br>· **배경이미지 생성** → Gemini Imagen (이미지 특화 모델)<br>· **카드 PNG 렌더링** → HTML + Playwright (결정적 렌더링) |
| **4) 실패 복구** | 자기 평가 **7점 미만** → 피드백과 함께 재생성 (최대 3회)<br>3회 모두 미달이면 최고 점수 버전 채택 → 구글 시트에서 Human이 최종 승인 |
| **5) 측정 · 비용** | 단계별 자기 평가 점수 · 재시도 횟수를 로그로 기록<br>세트당 API 비용 **약 $0.025**로 예측 가능 · 이상 시 상한 차단 |

# 결과물 확인

# 언제 어느 쪽을 써야 할까?

| 상황 (변동성 기준) | 유리한 접근 방식 |
|---|---|
| 고정 파이프라인 + 높은 처리량 | </> 코드 기반 |
| 자주 바뀌는 워크플로우 | 하네스 기반 |
| 비개발자 직접 튜닝 필요 | 하네스 기반 |
| 엄격한 SLA / 레거시 연동 | </> 코드 기반 |
| 빠른 실험 · 프로토타이핑 | 하네스 기반 |

## Humans steer. Agents execute.

코드로 풀면 `thumbnail_agent.py`를 짜야 하지만,
하네스로 풀면 `thumbnail-writer.md`에 **역할만 정의**하면 된다.

# 참고

- **zoey.day 의 자동화 하네스 아티클**
  - 1편 — Claude Code Agent : Product Page Localization
  - 2편 — Pipeline 편
  - 3편 — Image 편
- **jay.pm.ai 의 에이전트 구성기 사례** — *별도 첨부 PDF 참고*
- **하네스 구성 Claude Code 플러그인** : github.com/revfactory/harness
- **본 프로젝트 — 카드 뉴스 생성 자동화 하네스** : github.com/hyejungg/gen-card-news-harness

# 감사합니다.